INDUSTRY REPORT

Loading...

Macon-Bibb County Workforce Analysis

Loading workforce data...

TOTAL EMPLOYED	SHARE OF JOB MARKET
—	—
workers in this sector	of all local employment

AVERAGE ANNUAL WAGE	PROJECTED ANNUAL OPENINGS
—	—
across all sub-sectors	total demand openings

Share of the Local Economy

Employment distribution across all major industry sectors.

Breakdown by Employer Type

Top sub-sectors by total employment.

Highest & Lowest Paying Sub-Sectors

Average annual wage comparison across sub-sectors.

Sub-Sectors with the Highest Labor Demand

Projected annual job openings (new positions + replacement demand).

'} `; } // —— Boot

```
document.addEventListener('DOMContentLoaded', async () ⇒ { const params = new URLSearchParams(window.location.search); const id = params.get('id'); const config = INDUSTRIES[id]; if (!config) { document.getElementById('coverTitle').textContent = 'Industry Not Found'; document.getElementById('narrative').textContent = `No configuration found for id="${id}". Check the URL.`; return; } // Apply industry accent color to CSS variables const { r, g, b } = hexToRgb(config.color); document.documentElement.style.setProperty('--ind', config.color); document.documentElement.style.setProperty('--ind-dim', `rgba(${r},${g},${b},0.14)`); // Cover document.title = `${config.name} — Workforce Data`; document.getElementById('coverLabel').textContent = `Industry Report · ${config.naicsLabel}`; document.getElementById('coverTitle').textContent = config.name; // Load data let rows; try { const resp = await fetch('../Data/processed/Industry_Snapshot.csv'); if (!resp.ok) throw new Error(resp.status); ({ rows } = parseCSV(await resp.text())); } catch (e) { document.getElementById('narrative').textContent = 'Could not load data. Run via a local server: python3 -m http.server 8080'; return; } // Partition rows const totalRow = rows.find(r ⇒ (r['NAICS'] || '').trim() === ''); const totalEmpl = toNum(totalRow?.['Empl']); const indRows = rows.filter(r ⇒ { const code = (r['NAICS'] || '').trim(); return config.prefixes.some(p ⇒ code.startsWith(p)); }); if (indRows.length === 0) { document.getElementById('narrative').textContent = 'No data found for this industry in the current dataset.'; return; } // KPIs const indEmpl = indRows.reduce((s, r) ⇒ s + toNum(r['Empl']), 0); const indDemand = indRows.reduce((s, r) ⇒ s + toNum(r['Total Demand']), 0); const wageNum = indRows.reduce((s, r) ⇒ s + toNum(r['Avg Ann Wages']) * toNum(r['Empl']), 0); const wageDen = indRows.reduce((s, r) ⇒ s + toNum(r['Empl']), 0); const avgWage = wageDen > 0 ? wageNum / wageDen : 0; const share = totalEmpl > 0 ? (indEmpl / totalEmpl) * 100 : 0; // Sector rank (aggregate, apply NAICS_MERGE) const sectorMap = new Map(); rows.filter(r ⇒ (r['NAICS'] || '').trim() !== '').forEach(r ⇒ { const raw = (r['NAICS'] || '').slice(0, 2); const code = NAICS_MERGE[raw] || raw; sectorMap.set(code, (sectorMap.get(code) || 0) + toNum(r['Empl'])); }); const sectorsSorted = [...sectorMap.entries()].sort((a, b) ⇒ b[1] - a[1]); const canonicalId = NAICS_MERGE[id] || id; const rank = sectorsSorted.findIndex(([c]) ⇒ c === canonicalId) + 1; // Narrative document.getElementById('narrative').innerHTML = `${config.name} accounts for about ` + `${share.toFixed(1)}% of Macon-Bibb's job market, ` + `making it the ${ordinal(rank)} largest industry in the
```

```
region. ` + `Approximately ${fmtNum(indEmpl)} people work in this sector, ` + `with an average annual wage of ${fmtDollar(Math.round(avgWage))} across all sub-sectors.`; // KPI cards document.getElementById('kpi-empl').textContent = fmtNum(indEmpl); document.getElementById('kpi-share').textContent = share.toFixed(1) + '%'; document.getElementById('kpi-wage').textContent = fmtDollar(Math.round(avgWage)); document.getElementById('kpi-demand').textContent = fmtNum(indDemand); // Section sub-text (dynamic industry name) const shortName = config.name.split(',')[0].split('&')[0].trim(); document.getElementById('doughnut-sub').textContent = `Employment distribution across all major industry sectors. ${shortName} is highlighted; all other sectors are shown in muted tones.`; document.getElementById('empl-sub').textContent = `Top sub-sectors by total employment within ${config.name}.`; document.getElementById('wage-sub').textContent = `Top and bottom paying employer types by average annual wage within ${config.name}.`; document.getElementById('demand-sub').textContent = `Projected annual job openings (new positions + replacement demand) for the top sub-sectors within ${config.name}.`; wireBuilderLinks(id, config, shortName); // Store for theme-switch rebuilds _sectorsSorted = sectorsSorted; _canonicalId = canonicalId; _indRows = indRows; _accentColor = config.color; try { _charts.doughnut = buildDoughnut(sectorsSorted, canonicalId, config.color); } catch(e) { console.error('doughnut', e); } try { _charts.empl = buildEmplChart(indRows, config.color); } catch(e) { console.error('empl', e); } try { _charts.wage = buildWageChart(indRows, config.color); } catch(e) { console.error('wage', e); } try { _charts.demand = buildDemandChart(indRows, config.color); } catch(e) { console.error('demand', e); } }); // ——— Builder links ——————————————————————————————————————————————————————————— function wireBuilderLinks(id, config, shortName) { const base = '../index.html'; const filter = config.prefixes.join(','); // Shared dark theme matching the report aesthetic const darkTheme = { colorMode: 'gradient', chartBg: '#141416', textColor: '#d4d4d8', wordTextColor: '#d4d4d8', numberTextColor: '#52525b', gridColor: '#27272a', axisColor: '#27272a', showGrid: false, showBarLabels: true, overlayBarLabels: true, }; function buildURL(extra, themeExtra = {}) { const theme = encodeURIComponent(JSON.stringify({ ...darkTheme, ...themeExtra })); const pairs = Object.entries(extra).map(([k, v]) => `${k}=${encodeURIComponent(v)}`); return `${base}?${pairs.join('&')}&theme=${theme}`; } // Doughnut — sector share, level 2 aggregation, highlight this industry const doughnutHL = NAICS_NAMES[NAICS_MERGE[id] || id] || shortName; document.getElementById('builder-doughnut').href = buildURL({ source: 'Industry_Snapshot.csv', type: 'doughnut', level: '2', sort: 'desc', cols: 'Empl', title: 'Employment Share by Sector', }, { highlightSliceLabel: doughnutHL.toLowerCase(), highlightSliceColor: config.color, }); // Employment — top 20 sub-sectors by Empl document.getElementById('builder-empl').href = buildURL({ source: 'Industry_Snapshot.csv', type: 'horizontalBar', level: '6', filter: filter, sort: 'desc', cols: 'Empl', maxrows: '20', numfmt: 'auto', title: `${shortName} — Employment by Sub-Sector`, }, { gradientStart: config.color, gradientEnd: fadeHex(config.color, 0.35) }); // Wage — top 20 sub-sectors by avg wage document.getElementById('builder-wage').href = buildURL({ source: 'Industry_Snapshot.csv', type: 'horizontalBar', level: '6', filter: filter, sort: 'desc', cols: 'Avg Ann Wages', maxrows: '20', numfmt: 'dollar', title: `${shortName} — Average Annual Wage by Sub-Sector`, }, { gradientStart: config.color, gradientEnd: fadeHex(config.color, 0.35) }); // Demand — top 15 sub-sectors by Total Demand document.getElementById('builder-demand').href = buildURL({ source: 'Industry_Snapshot.csv', type: 'horizontalBar', level: '6', filter: filter, sort: 'desc', cols: 'Total Demand', maxrows: '15', numfmt: 'auto', title: `${shortName} — Projected Annual Openings by Sub-Sector`, }, { gradientStart: config.color, gradientEnd: fadeHex(config.color, 0.35) }); } function fadeHex(hex, alpha) { const { r, g, b } = hexToRgb(hex); return `rgba(${r},${g},${b},${alpha})`; } // ——— Doughnut
```

```
function buildDoughnut(sectorsSorted, highlightCode, accentColor) { const labels = sectorsSorted.map(([c]) => NAICS_NAMES[c] || c); const data = sectorsSorted.map(([, v]) => v); const total = data.reduce((a, b) => a + b, 0); const isDark = C.bg !== '#ffffff'; const colors = sectorsSorted.map(([c], i) => c === highlightCode ? accentColor : isDark ? `hsl(240,5%,${15 + (i * 0.5)}%)` : `hsl(220,8%,${82 - (i * 0.4)}%)` ); const mutedLabel = isDark ? 'rgba(255,255,255,0.35)' : 'rgba(0,0,0,0.35)'; // Legend document.getElementById('doughnutLegend').innerHTML = sectorsSorted.map(([c, v], i) => { const isHL = c === highlightCode; const pct = ((v / total) * 100).toFixed(1); return `
    ${NAICS_NAMES[c] || c}    ${pct}%
`; }).join(''); return new Chart(document.getElementById('doughnutChart'), { type: 'doughnut', data: { labels, datasets: [{ data, backgroundColor: colors, borderColor: C.bg, borderWidth: 2 }] }, options: { responsive: true, maintainAspectRatio: false, cutout: '60%', plugins: { legend: { display: false }, tooltip: { callbacks: { label: ctx => ` ${fmtNum(ctx.raw)} (${((ctx.raw/total)*100).toFixed(1)}%)` } }, datalabels: { display: ctx => (ctx.dataset.data[ctx.dataIndex] / total * 100) >= 5, formatter: (v, ctx) => { const pct = ((v / total) * 100).toFixed(0) + '%'; const clr = ctx.chart.data.datasets[0].backgroundColor[ctx.dataIndex]; const lbl = ctx.chart.data.labels[ctx.dataIndex]; return clr === accentColor ? `${lbl}\n${pct}` : pct; }, color: ctx => { const clr = ctx.chart.data.datasets[0].backgroundColor[ctx.dataIndex]; return clr === accentColor ? '#fff' : mutedLabel; }, font: { size: 10, weight: '600' }, textAlign: 'center', } } }, plugins: [bgPlugin, ChartDataLabels] }); } // ── Employment chart ──────────────────────────────────────
function buildEmplChart(indRows, accentColor) { const sorted = [...indRows] .filter(r => toNum(r['Empl']) > 0) .sort((a, b) => toNum(b['Empl']) - toNum(a['Empl'])) .slice(0, 20); if (sorted.length === 0) { document.getElementById('emplWrap').innerHTML = '

No employment data available for this sector.

'; return; } const { r, g, b } = hexToRgb(accentColor); const colors = sorted.map((_, i) => `rgba(${r},${g},${b},${0.85 - (i / sorted.length) * 0.45})`); const canvas = document.getElementById('emplChart'); canvas.parentElement.style.height = Math.max(280, sorted.length * 30 + 60) + 'px'; return new Chart(canvas, { type: 'bar', data: { labels: sorted.map(r => truncate(r['Industry'], 44)), datasets: [{ data: sorted.map(r => toNum(r['Empl'])), backgroundColor: colors, borderWidth: 0, barPercentage: 0.72, categoryPercentage: 1 }] }, options: { indexAxis: 'y', responsive: true, maintainAspectRatio: false, layout: { padding: { right: 58, top: 4, bottom: 4 } }, plugins: { legend: { display: false }, tooltip: { callbacks: { label: ctx => ' ' + fmtNum(ctx.raw) + ' workers' } }, datalabels: { anchor:'end', align:'right', clamp:false, clip:false, color:C.muted, font:{ size:10, weight:'500' }, formatter: fmtNum } }, scales: { y: { grid:{ display:false }, border:{ display:false }, ticks:{ color:C.text, font:{ size:11 } } }, x: { display: false } } }, plugins: [bgPlugin, ChartDataLabels] }); } // ── Wage chart
──────────────────────────────────────── function buildWageChart(indRows, accentColor) { const withWages = [...indRows] .filter(r => toNum(r['Avg Ann Wages']) > 0 && toNum(r['Empl']) > 0) .sort((a, b) => toNum(b['Avg Ann Wages']) - toNum(a['Avg Ann Wages'])); if (withWages.length === 0) { document.getElementById('wageWrap').innerHTML = '
```

No wage data available for this sector.

'; return; } // Adaptive: top N and bottom N based on available rows const half = Math.min(10, Math.max(3, Math.floor(withWages.length / 2))); const top = withWages.slice(0, half); const bot = [...withWages].reverse().slice(0, half).reverse(); // Deduplicate if overlap (small sectors) const topSet = new Set(top.map(r ⇒ r['NAICS'])); const botDed = bot.filter(r ⇒ !topSet.has(r['NAICS'])); const combined = [...top, ...botDed]; const topColor = accentColor; const botColor = '#f59e0b'; const colors = combined.map((_, i) ⇒ i < top.length ? fadeColor(topColor, 0.8) : fadeColor(botColor, 0.8)); const canvas = document.getElementById('wageChart'); canvas.parentElement.style.height = Math.max(200, combined.length * 30 + 80) + 'px'; const chart = new Chart(canvas, { type: 'bar', data: { labels: combined.map(r ⇒ truncate(r['Industry'], 44)), datasets: [{ data: combined.map(r ⇒ toNum(r['Avg Ann Wages'])), backgroundColor: colors, borderWidth: 0, barPercentage: 0.72, categoryPercentage: 1 }] }, options: { indexAxis: 'y', responsive: true, maintainAspectRatio: false, layout: { padding: { right: 68, top: 4, bottom: 4 } }, plugins: { legend: { display: false }, tooltip: { callbacks: { label: ctx ⇒ ' Avg. ' + fmtDollar(ctx.raw) + ' / year' } }, datalabels: { anchor:'end', align:'right', clamp:false, clip:false, color: ctx ⇒ ctx.dataIndex < top.length ? accentColor : '#f59e0b', font: { size: 10, weight: '600' }, formatter: fmtDollar } }, scales: { y: { grid:{ display:false }, border:{ display:false }, ticks:{ color:C.text, font:{ size:11 } } }, x: { display: false } } }, plugins: [bgPlugin, ChartDataLabels] }); // Color legend const wrap = document.getElementById('wageChart').closest('.chart-wrap'); const leg = document.createElement('div'); leg.className = 'wage-legend'; leg.innerHTML = `Highest Paying` + `Lowest Paying`; wrap.appendChild(leg); return chart; } // —— Demand chart —— function buildDemandChart(indRows, accentColor) { const sorted = [...indRows] .filter(r ⇒ toNum(r['Total Demand']) > 0) .sort((a, b) ⇒ toNum(b['Total Demand']) - toNum(a['Total Demand'])) .slice(0, 15); if (sorted.length === 0) { document.getElementById('demandWrap').innerHTML = '

No demand data available for this sector.

'; return; } const { r, g, b } = hexToRgb(accentColor); const colors = sorted.map((_, i) ⇒ `rgba(${r},${g},${b},${0.75 - (i / sorted.length) * 0.35})`); const canvas = document.getElementById('demandChart'); canvas.parentElement.style.height = Math.max(240, sorted.length * 30 + 60) + 'px'; return new Chart(canvas, { type: 'bar', data: { labels: sorted.map(r ⇒ truncate(r['Industry'], 44)), datasets: [{ data: sorted.map(r ⇒ toNum(r['Total Demand'])), backgroundColor: colors, borderWidth: 0, barPercentage: 0.72, categoryPercentage: 1 }] }, options: { indexAxis: 'y', responsive: true, maintainAspectRatio: false, layout: { padding: { right: 48, top: 4, bottom: 4 } }, plugins: { legend: { display: false }, tooltip: { callbacks: { label: ctx ⇒ ' ' + fmtNum(ctx.raw) + ' projected openings' } }, datalabels: { anchor:'end', align:'right', clamp:false, clip:false, color:C.muted, font:{ size:10, weight:'500' }, formatter: fmtNum } }, scales: { y: { grid:{ display:false }, border:{ display:false }, ticks:{ color:C.text, font:{ size:11 } } }, x: { display: false } } }, plugins: [bgPlugin, ChartDataLabels] }); }